

Documentacao do Sistema Beelbem Motoboy

Atualizado em: 23/05/2026

Este documento explica, em linguagem pratica, o que existe em cada parte do projeto. A ideia e ajudar voce a saber onde pedir alteracoes: tela da loja, tela do motoboy, validacoes, banco de dados, funcoes do Supabase, notificacoes e regras de entrega.

1. Visao Geral

O Beelbem Motoboy e um web app feito com React, Vite e Supabase. Ele tem quatro perfis principais:

- `system_admin` : administra tudo.
- `city_admin` : administra uma cidade.
- `store_admin` : lojista, cria entregas e acompanha corridas.
- `courier_admin` : motoboy, fica online/offline, recebe ofertas e aceita entregas.

O frontend roda no navegador. O backend principal e o Supabase, que cuida de:

- autenticacao de usuarios;
- tabelas do banco;
- regras de seguranca com RLS;
- funcoes Edge;
- storage para documentos;
- realtime para atualizacoes automaticas;
- push notification para motoboy.

2. Como o Sistema Abre

Arquivo principal:

- `src/main.jsx` : ponto de entrada do React. Ele carrega o componente `App` dentro do elemento `#root` do `index.html`.
- `src/App.jsx` : controla a sessao, o perfil logado, a cidade selecionada e decide qual tela exibir.

Fluxo simplificado:

1. O navegador abre `index.html`.
2. `src/main.jsx` renderiza `App`.
3. `App` verifica a sessao Supabase.
4. `App` busca o perfil na tabela `profiles`.
5. Conforme o `role`, manda o usuario para: - admin; - loja; - motoboy; - login/publico.

3. Arquivos da Raiz

`package.json`

Define o projeto, scripts e dependencias.

Scripts importantes:

- `npm run dev` : roda o projeto localmente.
- `npm run build` : gera a versao de producao na pasta `dist`.
- `npm run preview` : testa localmente a build gerada.

Dependencias principais:

- `react` e `react-dom` : interface.
- `vite` : build e servidor local.
- `@supabase/supabase-js` : comunicacao com Supabase.
- `lucide-react` : icones.
- `zxcvbn` : forca de senha.
- `supabase` : CLI do Supabase.

`package-lock.json`

Trava as versoes exatas das dependencias instaladas.

`index.html`

HTML base do Vite. Define titulo, icones e o elemento onde o React entra.

`vercel.json`

Configura deploy no Vercel:

- build: `npm run build`;
- pasta publicada: `dist`;
- rewrites para enviar qualquer rota para `index.html`.

`.env.example`

Modelo das variaveis de ambiente.

`.env.local` e `.env.production`

Configuracoes reais de ambiente. Normalmente contem chaves como URL e anon key do Supabase. Evite expor em prints ou documentos publicos.

`.gitignore`

Diz ao Git quais arquivos ou pastas ignorar.

4. Pasta `public`

Arquivos servidos diretamente no site.

`public/beelbem-icon.png`

Icone usado no app, manifesto e notificacoes.

`public/login-hero.png`

Imagem usada na tela de login.

`public/manifest.webmanifest`

Configura o app como PWA/mobile:

- nome;
- icones;
- cores;
- modo de exibicao.

`public/sw.js`

Service Worker. Ajuda com cache e notificacoes no navegador. Importante para push notifications e comportamento mobile.

5. Pasta `src`

Codigo principal do frontend.

`src/main.jsx`

Renderiza o React:

- importa estilos globais;
- importa `App` ;
- monta a aplicacao no DOM.

`src/App.jsx`

E o roteador e controlador principal do app.

Responsabilidades:

- ler a rota pelo hash da URL, como `#login`, `#courier-home`, `#store-home`;
- verificar sessao do Supabase;
- buscar perfil em `profiles`;
- carregar cidades, lojas e motoboys;
- calcular metricas da cidade;
- decidir qual tela mostrar conforme o papel do usuario.

Quando alterar:

- problemas de redirecionamento;
- usuario indo para tela errada;
- carregamento inicial;
- listas globais de cidades, lojas e motoboys;
- regras de acesso por perfil no frontend.

`src/supabaseClient.js`

Cria o cliente Supabase usado no frontend.

Usa:

- `VITE_SUPABASE_URL` ;
- `VITE_SUPABASE_ANON_KEY` .

Tambem exporta `supabaseConfigStatus` , usado para saber se o Supabase esta configurado.

Quando alterar:

- troca de projeto Supabase;
- problemas de conexao;
- configuracao global de auth.

`src/styles.css`

Arquivo grande com todos os estilos visuais do sistema.

Contem estilos para:

- login;
- admin;
- loja;
- motoboy;
- mapas;
- modais;
- cards;
- tabelas;

- mobile/responsivo.

Quando alterar:

- cores;
- tamanhos;
- layout;
- botoes;
- espaco entre elementos;
- responsividade.

6. Layouts

`src/layouts/LayoutAdmin.jsx`

Layout base das telas administrativas.

`src/layouts/LayoutLojista.jsx`

Layout base das telas do lojista.

`src/layouts/LayoutMotoboy.jsx`

Layout base das telas do motoboy.

Esses arquivos geralmente servem como embrulho visual. Se o problema for cabeçalho, fundo, estrutura geral ou container de uma area, vale olhar aqui.

7. Telas Publicas

Pasta: `src/paginas/publico`

`AuthPublic.jsx`

Contem telas e fluxos de autenticacao:

- login;
- lembrar login;
- esqueci minha senha;
- criar conta;
- criar senha a partir de link;
- validacao de forca de senha.

Componentes/funcoes importantes:

- `LoginView` : tela de login.

- `ForgotPasswordView` : recuperacao de senha.
- `CreateAccountView` : criacao publica de conta.
- `CreatePasswordView` : criacao/alteracao de senha por link.

Quando alterar:

- texto da tela de login;
- campos de login;
- regras de senha;
- mensagens de erro de autenticacao;
- fluxo de senha inicial.

`AuthUnavailableView.jsx`

Tela exibida quando o Supabase nao esta configurado.

`JoinView.jsx`

Tela publica para escolha/caminho de cadastro.

`PublicSignupView.jsx`

Cadastro publico de loja ou motoboy.

Responsabilidades:

- escolher cidade;
- validar CNPJ/CPF, e-mail, telefone, CEP;
- enviar pre-cadastro para a Edge Function `public-signup`;
- deixar cadastro pendente para analise.

Quando alterar:

- campos do pre-cadastro;
- mensagens do cadastro publico;
- validacoes iniciais de loja/motoboy.

8. Telas Administrativas

Pasta: `src/paginas/admin`

`AdminShell.jsx`

Estrutura visual principal do admin:

- menu lateral;
- navegacao entre paginas administrativas;

- cabecalho;
- botao sair.

AdminPages.jsx

Arquivo grande com quase todas as telas administrativas.

Contem:

- dashboard/visao geral;
- mapa;
- cidades;
- lojas;
- central de lojas;
- motoboys;
- central de motoboys;
- usuarios/acessos;
- formularios de cadastro e edicao.

Responsabilidades comuns:

- criar e editar cidades;
- criar e editar lojas;
- criar e editar motoboys;
- gerar convite/senha de acesso;
- ativar/desativar cadastros;
- abrir arquivos de documentos no storage;
- trocar cidade selecionada;
- visualizar status de lojas e entregadores.

Quando alterar:

- formulario de loja;
- formulario de motoboy;
- campos administrativos;
- telas de centro/relatorio;
- permissoes visuais no painel admin;
- textos e botoes do painel administrativo.

9. Tela do Lojista

Pasta: `src/paginas/lojista`

InicioLojista.jsx

Arquivo simples de entrada/compatibilidade para a area do lojista.

StoreHomeView.jsx

Tela principal da loja.

Responsabilidades:

- mostrar status da loja: aberta/fechada;
- permitir abrir/fechar loja;
- atualizar dados da loja;
- criar entrega manualmente;
- criar entrega por foto/comprovante;
- chamar Gemini para analisar comprovante;
- validar horario previsto e taxa;
- acompanhar entregas do dia;
- acompanhar entregas ao vivo;
- mostrar status: a caminho da loja, a caminho do cliente, atrasadas;
- exibir popup quando uma entrega foi aceita;
- monitorar entregas pendentes sem aceite.

Fluxo de criar entrega manual:

1. Usuario abre "Realizar pedido". 2. Preenche pedido, cliente, telefone, horario, taxa e endereco. 3. `validarPedidoLoja` valida horario e taxa. 4. `createDeliveryWithQueue` cria cliente, entrega e fila. 5. Sistema oferece a entrega aos motoboys online elegiveis.

Fluxo de criar entrega por foto:

1. Lojista envia foto. 2. `analisarComprovantePedidoComGemini` tenta extrair dados. 3. `validar_dadoscomprovante_gemini` verifica se os campos foram encontrados. 4. Dados extraídos preenchem o formulario. 5. Lojista confirma a entrega.

Quando alterar:

- tela e campos do lojista;
- criacao de pedidos;
- validacao visual do formulario;
- comportamento de loja aberta/fechada;
- analise de comprovante;
- acompanhamento de entregas.

10. Tela do Motoboy

Pasta: `src/paginas/motoboy`

`InicioMotoboy.jsx`

Arquivo simples de entrada/compatibilidade para a area do motoboy.

`CourierHomeView.jsx`

Tela principal do motoboy.

Responsabilidades:

- motoboys online; - lojas abertas; - entregas do dia;
- exibir nome do motoboy;
- mostrar status online/offline;
- perguntar se quer ficar online quando estiver offline;
- mudar disponibilidade;
- receber entrega disponivel;
- aceitar ou recusar entrega;
- mostrar contagem regressiva da oferta;
- expirar oferta sem resposta;
- confirmar retirada na loja;
- confirmar entrega ao cliente;
- mostrar pontos/XP;
- acompanhar indicadores:
- assinar Realtime do Supabase para fila e indicadores;
- atualizar localizacao do motoboy;
- registrar push notification.

Fluxo de oferta:

1. A tela escuta `delivery_queue` para o motoboy logado. 2. Quando existe linha `offered`, busca a entrega. 3. Mostra modal de nova entrega. 4. Se aceitar, chama `acceptQueuedDelivery`. 5. Se recusar, chama `rejectQueuedDelivery`. 6. Se passar do tempo, chama `expireQueuedDeliveryOffer`.

Quando alterar:

- experiencia do motoboy;
- textos de aceite/recusa;
- comportamento online/offline;
- tempo da oferta;
- mapa do motoboy;

- indicadores superiores;
- notificacoes no navegador.

11. Regras de Entrega e Fila

`src/cadastra_entrega.js`

E um dos arquivos mais importantes do sistema.

Responsabilidades:

- formatar entrega para a tela do motoboy;
- criar entrega com fila;
- montar fila de motoboys elegiveis;
- ordenar motoboys por quantidade de entregas, distancia e data de cadastro;
- buscar proxima entrega do motoboy;
- aceitar entrega;
- recusar entrega;
- expirar oferta;
- confirmar retirada;
- confirmar entrega finalizada;
- atualizar disponibilidade do motoboy;
- atualizar localizacao;
- coordenadas de links do Google Maps;
- chamar notificacao de oferta.

Funcoes importantes:

- `createDeliveryWithQueue` : cria cliente, entrega e fila.
- `buildDeliveryQueue` : seleciona motoboys disponiveis e monta `delivery_queue`.
- `getNextDeliveryForCourier` : busca entrega aceita ou oferta aberta para um motoboy.
- `acceptQueuedDelivery` : primeiro motoboy que aceita assume a entrega.
- `rejectQueuedDelivery` : registra recusa.
- `expireQueuedDeliveryOffer` : expira oferta sem resposta.
- `notifyNextCourierOffer` : ativa/oferece entrega na fila.
- `setCourierAvailable` : muda online/offline.
- `updateCourierLocation` : salva latitude/longitude do motoboy.

Observacao importante sobre a fila atual:

- A fila ainda existe em `delivery_queue`.
- Ela registra quais motoboys eram elegiveis.

- A entrega pode ser oferecida para varios motoboys online ao mesmo tempo.
- Quem aceitar primeiro muda `deliveries.status` de `pending` para `assigned`.
- Os outros nao conseguem aceitar depois, porque a entrega nao esta mais pendente.

Quando alterar:

- regra de distribuicao;
- quem recebe entrega;
- aceite simultaneo;
- expiracao;
- recusa;
- fila;
- criterios de distancia e prioridade.

`src/regras_localizacao.js`

Regras matematicas de localizacao.

Responsabilidades:

- validar coordenadas;
- calcular distancia aproximada em km;
- calcular diferenca de direcao;
- verificar se motoboy esta perto da loja;
- estimar tempo por distancia;
- medir proximidade de um ponto com uma rota.

Quando alterar:

- raio de proximidade;
- calculo de distancia;
- regras de localizacao e rota.

`src/regras_agrupamento_entregas.js`

Regras para avaliar entregas compatíveis com uma corrida em andamento.

Responsabilidades:

- verificar limite de entregas simultaneas;
- verificar se nova entrega combina com rota atual;
- calcular grau de compatibilidade;
- sugerir ordem de coleta/entrega;
- rejeitar entregas que aumentam demais a rota.

Quando alterar:

- entregas agrupadas;
- limite de entregas simultaneas;
- criterio de compatibilidade;
- ordem sugerida de rota.

`src/ValidaPedidoLoja.js`

Validacoes do pedido criado pela loja.

Responsabilidades:

- validar horario previsto;
- impedir horario menor que 10 minutos;
- impedir horario maior que 12 horas;
- validar taxa minima e maxima;
- converter valor monetario.

Constantes importantes:

- `MINUTOS_MINIMOS_HORARIO_PREVISTO = 10`
- `MINUTOS_MAXIMOS_HORARIO_PREVISTO = 720`
- `VALOR_MINIMO_ENTREGA = 5`
- `VALOR_MAXIMO_ENTREGA = 100`

Quando alterar:

- limite de horario;
- taxa minima;
- taxa maxima;
- mensagens de validacao do pedido.

`src/xp_motoboy.js`

Regras de pontos/XP do motoboy.

Responsabilidades:

- inserir eventos de XP;
- somar pontos no `courier_points`;
- dar XP por aceitar entrega;
- dar XP por retirar pedido;
- dar XP por entrega no prazo.

Quando alterar:

- pontuacao;
- motivos de XP;

- regras de recompensa.

12. Analise de Comprovante e Gemini

`src/analisa_comprovante_pedido.js`

Integra com Gemini para ler foto de comprovante/pedido.

Responsabilidades:

- converter imagem para base64;
- montar prompt para Gemini;
- chamar API do Google Gemini;
- tentar modelo fallback;
- extrair JSON da resposta;
- normalizar dados;
- transformar analise em pedido da loja;
- fornecer modo de teste com texto de comprovante iFood.

Quando alterar:

- prompt de OCR;
- modelo Gemini;
- campos extraídos da foto;
- tratamento de erro da IA.

`src/valid_dadoscomprovante_gemini.js`

Valida se o Gemini encontrou os campos esperados.

Responsabilidades:

- identificar campos nao encontrados;
- montar mensagem de aviso;
- bloquear quando nenhum dado util foi extraido.

Quando alterar:

- quais campos sao obrigatorios na analise;
- mensagens de falha da leitura da foto.

13. Utilitarios

`src/utils/validators.js`

Funcoes gerais de validacao e mascaras.

Contem:

- CPF;
- CNPJ;
- CEP;
- e-mail;
- telefone;
- mascaras de documento;
- validacao de formularios de loja;
- validacao de usuarios de acesso;
- validacao de motoboy;
- forza de senha com `zxcvbn`.

Quando alterar:

- regras de CPF/CNPJ/telefone;
- campos obrigatorios dos formularios;
- politica de senha.

`src/utils/pageRouting.js`

Define rotas por perfil.

Responsabilidades:

- converter hash da URL em pagina;
- decidir tela inicial pelo role.

Quando alterar:

- rota inicial de cada perfil;
- nomes dos hashes da URL.

14. Supabase: Banco de Dados

`supabase/schema.sql`

Arquivo principal do banco.

Contem:

- criacao das tabelas;
- colunas;
- constraints;
- indexes;
- triggers;
- funcoes auxiliares;
- RLS;
- policies;
- storage policies;
- publicacao realtime.

Tabelas principais:

- `cities` : cidades atendidas.
- `stores` : lojas.
- `couriers` : motoboys.
- `courier_points` : pontuacao total do motoboy.
- `profiles` : perfil de acesso ligado ao usuario auth.
- `access_invites` : convites de acesso.
- `customers` : clientes finais.
- `deliveries` : entregas.
- `delivery_events` : historico/status de entregas.
- `courier_xp_events` : eventos de XP.
- `delivery_rejections` : recusas de entrega.
- `delivery_queue` : fila/lista de motoboys elegiveis por entrega.
- `courier_push_subscriptions` : inscricoes de push notification.
- `system_settings` : configuracoes gerais.
- `audit_logs` : auditoria de alteracoes.

Status importantes:

`deliveries.status` pode incluir:

- `pending` ;
- `assigned` ;
- `picked_up` ;
- `on_route` ;
- `delivered` ;
- `cancelled` .

`delivery_queue.status` pode incluir:

- `waiting ;`
- `offered ;`
- `accepted ;`
- `rejected ;`
- `skipped ;`
- `expired .`

Quando alterar:

- novas tabelas;
- novas colunas;
- regras de seguranca;
- realtime;
- indices;
- triggers;
- storage.

`supabase/reset_access.sql`

Script para recriar/garantir usuario administrador do sistema.

Use com cuidado, porque mexe em acesso administrativo.

`supabase/public_signup_policies.sql`

Policies para permitir cadastro publico controlado de loja/motoboy.

`supabase/liberar_oferta_para_todos_motoboys_online.sql`

Script simples criado para remover a trava antiga que permitia apenas um motoboy com oferta ativa por entrega.

Conteudo:

```
drop index if exists public.delivery_queue_one_offered_per_delivery_idx;
```

15. Supabase: Migrations

Pasta: `supabase/migrations`

Cada arquivo registra uma mudanca incremental no banco.

`001_add_cities.sql`

Cria estrutura inicial de cidades.

002_access_control.sql

Adiciona controle de acesso/perfis.

003_store_type.sql

Adiciona/ajusta tipo de loja.

004_store_full_registration.sql

Expande cadastro completo de loja.

005_store_form_cleanup.sql

Ajustes de campos do formulario de loja.

006_access_user_documents.sql

Inclui documentos de usuario/acesso.

007_access_activation_and_password_invite.sql

Fluxo de ativacao e convite de senha.

008_password_setup_expiration.sql

Expiracao de link/configuracao de senha.

009_courier_full_registration.sql

Cadastro completo do motoboy.

010_courier_birth_date.sql

Campo de nascimento do motoboy.

011_courier_push_subscriptions.sql

Tabela de inscricoes push do motoboy.

012_courier_locations.sql

Tabela/localizacao do motoboy.

013_store_read_courier_points.sql

Permite loja ler pontos de motoboy quando necessario.

014_delivery_offer_rotation.sql

Indices/estrutura para rotacao de oferta.

015_single_active_delivery_offer.sql

Criou a regra antiga de apenas uma oferta ativa por entrega. Depois foi necessario remover essa trava para oferta simultanea.

016_advance_delivery_offer_rpc.sql

Cria RPC para avancar oferta na fila.

017_fix_advance_delivery_offer_rpc_ambiguity.sql

Corrige ambiguidade na RPC.

018_rename_advance_offer_rpc_courier_output.sql

Ajusta nome/retorno da RPC.

019_loop_delivery_offer_queue_after_rejection.sql

Permite repetir/continuar fila apos recusas/expiracoes.

020_enable_courier_stats_realtime.sql

Habilita realtime para tabelas usadas nos indicadores do motoboy.

16. Supabase: Edge Functions

Pasta: `supabase/functions`

complete-password-setup/index.ts

Marca/valida conclusao da criacao de senha.

create-courier-invite/index.ts

Cria ou atualiza motoboy e usuario auth correspondente.

Responsabilidades:

- validar permissao;
- criar/atualizar `couriers`;
- criar/atualizar usuario no Supabase Auth;
- criar/atualizar `profiles`;
- devolver senha temporaria quando aplicavel.

manage-access-user/index.ts

Gerencia usuarios de acesso.

Pode:

- atualizar perfil;
- gerar nova senha;
- remover/desativar usuario dependendo da acao.

notify-courier-offer/index.ts

Envia push notification de nova entrega para motoboy.

Responsabilidades:

- buscar ofertas na `delivery_queue` ;
- marcar oferta como `offered` ;
- buscar inscricoes em `courier_push_subscriptions` ;
- montar payload;
- enviar Web Push;
- desativar inscricoes vencidas.

Quando alterar:

- notificacao push;
- mensagem da notificacao;
- comportamento em segundo plano;
- envio para um ou varios motoboys.

public-signup/index.ts

Recebe cadastro publico de loja ou motoboy.

Cria registro pendente/inativo para analise posterior.

reset-system-admin/index.ts

Funcao para recriar/resetar usuario administrador do sistema.

Use com cuidado.

send-access-invite/index.ts

Envia/processa convite de acesso:

- cria usuario Auth;
- cria perfil;
- marca convite como enviado;

- gera senha temporaria.

17. Fluxo Completo de Uma Entrega

1. Loja abre a tela `StoreHomeView`. 2. Loja cria pedido manual ou por foto. 3. `ValidaPedidoLoja.js` valida horario e taxa. 4. `createDeliveryWithQueue` cria: - `customers`; - `deliveries`; - `delivery_queue`. 5. `buildDeliveryQueue` busca motoboys: - da cidade; - ativos; - aprovados; - online (`availability_status = available`). 6. Sistema registra ordem em `queue_position`. 7. Sistema marca ofertas como `offered`. 8. Cada motoboy online ve a entrega na `CourierHomeView`. 9. Quem aceitar primeiro chama `acceptQueuedDelivery`. 10. `deliveries.status` muda de `pending` para `assigned`. 11. Outros motoboys nao conseguem assumir a mesma entrega. 12. Motoboy confirma retirada. 13. Motoboy confirma entrega. 14. Sistema registra eventos e XP.

18. Onde Pedir Alteracoes

Para mudar tela do lojista:

- `src/paginas/lojista/StoreHomeView.jsx`
- `src/styles.css`

Para mudar tela do motoboy:

- `src/paginas/motoboy/CourierHomeView.jsx`
- `src/styles.css`

Para mudar fila/oferta/aceite:

- `src/cadastra_entrega.js`
- `supabase/functions/notify-courier-offer/index.ts`
- `supabase/schema.sql`
- migrations em `supabase/migrations`

Para mudar validacao de pedido:

- `src/ValidaPedidoLoja.js`

Para mudar validacao de CPF/CNPJ/telefone/senha:

- `src/utils/validators.js`

Para mudar leitura de foto/IA:

- `src/analisa_comprovante_pedido.js`
- `src/valid_dadoscomprovante_gemini.js`

Para mudar login/senha:

- `src/paginas/publico/AuthPublic.jsx`

- Edge Functions de acesso em `supabase/functions`

Para mudar banco:

- `supabase/schema.sql`
- `supabase/migrations`

Para mudar deploy:

- `vercel.json`
- `package.json`

19. Cuidados Antes de Alterar

- Sempre rodar `npm run build` depois de mudar frontend.
- Se mudar banco, aplicar SQL no Supabase remoto.
- Se mudar Edge Function, precisa deploy da function no Supabase.
- Se mudar frontend, push para `main` dispara Vercel.
- Cuidado com `.env.local` e `.env.production`, pois podem conter chaves.
- Nao apagar migrations antigas sem entender o historico.
- Se uma entrega antiga estiver com estado antigo na fila, teste com entrega nova.

20. Resumo Rapido por Arquivo

Arquivo	O que faz
<code>src/main.jsx</code>	Inicia o React.
<code>src/App.jsx</code>	Controla sessao, perfil, rotas e tela exibida.
<code>src/supabaseClient.js</code>	Configura cliente Supabase.
<code>src/styles.css</code>	Estilos visuais do app inteiro.
<code>src/paginas/publico/AuthPublic.jsx</code>	Login, senha, cadastro de conta.
<code>src/paginas/publico/PublicSignupView.jsx</code>	Pre-cadastro publico de loja/motoboy.
<code>src/paginas/admin/AdminShell.jsx</code>	Estrutura do painel admin.
<code>src/paginas/admin/AdminPages.jsx</code>	Telas administrativas.
<code>src/paginas/lojista/StoreHomeView.jsx</code>	Tela principal da loja.
<code>src/paginas/motoboy/CourierHomeView.jsx</code>	Tela principal do motoboy.
<code>src/cadastra_entrega.js</code>	Criacao, fila, aceite, recusa e conclusao de entregas.
<code>src/ValidaPedidoLoja.js</code>	Validacao de horario e taxa do pedido.

Arquivo	O que faz
src/regras_localizacao.js	Distancia, coordenadas e proximidade.
src/regras_agrupamento_entregas.js	Compatibilidade de entregas agrupadas.
src/analisa_comprovante_pedido.js	Analise de foto com Gemini.
src/valid_dadoscomprovante_gemini.js	Valida retorno da IA.
src/xp_motoboy.js	Pontos/XP do motoboy.
src/utils/validators.js	Validadores e mascaras.
src/utils/pageRouting.js	Roteamento por perfil.
supabase/schema.sql	Estrutura completa do banco.
supabase/migrations/*	Historico incremental de mudancas do banco.
supabase/functions/*	Funcoes backend no Supabase.
public/sw.js	Service worker e notificacoes.
public/manifest.webmanifest	Configuracao PWA.
vercel.json	Deploy Vercel.